Three-Tool Overlap Decomposition of furqan-lint Findings Across 25 Python Codebases

Bilal Syed Arfeen
BayyinahEnterprise

November 2026

Abstract

We decompose the findings of three static analysis tools -- furqan-lint v2.1.0 (structural honesty), bandit 1.9.4 (security), and a 30-rule subset of ruff 0.16.9 (style and convention) -- across 25 open-source Python packages (2.3M lines, 5,252 files). Using file-and-line co-location (window of 2 lines), we classify each of furqan-lint's 24,933 findings as not co-located with either other tool (45.5%, 11,338), co-located with ruff only (38.0%), co-located with bandit only (1.1%), or co-located with both (15.5%). SI-001 (Any annotation, which has a direct ruff counterpart in ANN401) accounts for 48.4% of the not-co-located findings; four rules plus the five-rule RC family -- kwargs passthrough (SI-005), hasattr masking (SI-007), empty except blocks (MR-005), the RC resource-lifecycle family, and try-except-continue (MR-007) -- account for 46.1%, though MR-005 and RC-001 have partial counterparts in ruff/bandit. An exploratory keyword-based classifier (not blind human review) applied to a stratified sample of 585 suggests that the not-co-located findings are predominantly code-health signals (~77.6% stratum-weighted) rather than safety-critical (~5.3%). Precision is not measured: the classifier cannot detect false positives for most rules. We provide raw JSON from all three tools for independent verification.

Note on terminology. Finding means a pattern flagged by a tool's rule at a specific file and line. Not co-located means no other tool produced a finding within 2 lines on the same file; this is a proxy for uniqueness, not proof of it (see Section 2.5). Precision (false-positive rate) is not measured in this study.

---

1. Introduction

Static analysis tools for Python are well-established in three categories: style and convention (ruff, flake8, pylint), type checking (mypy, pyright), and security scanning (bandit, semgrep). A practitioner adding a new tool to their CI pipeline needs to know: does it find anything the existing tools miss?

furqan-lint is a multi-language static analysis tool organized around *structural honesty* -- whether code's interface claims match its implementation behavior. An earlier finding-count comparison (Arfeen, 2026) showed that furqan-lint produces more findings than bandit in 22 of 25 packages, but did not decompose the overlap. This paper addresses that gap. It does not measure precision: the exploratory classification in Section 3.3 is a keyword heuristic, not a measurement.

We ask two questions:

1. **Overlap.** How much of furqan-lint's output is already caught by bandit, a 30-rule ruff subset, or both?
2. **Composition.** What rules drive the not-co-located findings, and what do they detect?

2. Method

2.1 Tools and configuration

Tool	Version	Configuration	Role
furqan-lint	v2.1.0	Default (39 active Python rules)	Structural honesty
bandit	1.9.4	Default (all rules)	Security scanning
ruff	0.16.9	30 rules mapped to furqan equivalents	Style + convention + security

ruff was run with a targeted rule set: 30 ruff rules with known furqan-lint equivalents (ANN401, BLE001, E722, FIX001/002, PGH001/002/003, S101, S104-S110, S301-S303, S307, S311, S324, S501, S602/603/605/606/608, TD001-003). This understates the overlap: additional ruff rules not included in the selection (SIM115, B006, PGH004, S112, S502-504, S506, S313-320) have partial or full equivalents to furqan-lint rules. Running ruff with its full 800+ rule set could increase the observed co-location rate (no full-rule comparison was performed). The 30-rule test provides a lower bound on ruff overlap, not an upper bound.

Neither ruff's `--ignore-noqa` nor bandit's `--ignore-nosec` was used, so upstream-suppressed lines (where the original developer added `# noqa` or `# nosec`) appear as not-co-located furqan-lint findings even when a counterpart rule exists.

2.2 Corpus

25 packages from PyPI, scanned in installed form (site-packages). Domain diversity: web frameworks, HTTP clients, data science, ORMs, task queues, templating, cryptography, SSH, scraping, CLIs, cloud SDKs, image processing. Total: 2,327,868 lines, 5,252 files. Installed packages include test files and, for numpy, 76 bundled C/C++/header files.

2.3 Overlap decomposition

For each furqan-lint finding, we check whether bandit or ruff produced a finding on the same file within 2 lines. A furqan-lint finding is classified as:

- **UNIQUE**: no bandit or ruff finding within 2 lines on the same file
- **RUFF_ONLY**: co-located with ruff but not bandit
- **BANDIT_ONLY**: co-located with bandit but not ruff
- **BOTH**: co-located with both

Co-location is a proxy for overlap, not proof of semantic equivalence. Two findings on the same line may detect unrelated patterns (coincidental co-location). The opposite also occurs: two tools may detect the same pattern on different lines (missed semantic overlap). The window of 2 lines was chosen as a balance; sensitivity is reported in Section 3.1.

The decomposition is one-directional: we iterate furqan-lint findings and check whether bandit/ruff co-locate. The reverse (bandit or ruff findings that furqan-lint misses) is not decomposed.

2.4 Exploratory classification

From the UNIQUE pool (11,338 findings), we drew a disproportionate stratified sample: 50 per rule with 100+ unique findings, min(20, n) per rule with 10-99 unique findings, all for rules with fewer than 10. Total sample: 585 findings across 33 rules (seed 42). Small strata (TB-*, RC-003, RC-004) are censuses or near-censuses; large strata (SI-001, SI-005) are sampled at <1%.

For each sampled finding, a deterministic keyword function read 11 lines of source context and answered two questions:

- **Q1 (pattern present):** Does the surrounding context contradict the rule's claim? Only 3 rules (RC-001, TB-018, MR-002) have contradiction checks. For the other 30 rules, Q1 is true by construction -- the function cannot detect false positives. The reported 581/585 "true positive" rate is therefore not a precision measurement.
- **Q2 (category):** A keyword-proximity heuristic assigns SAFETY, HEALTH, or DESIGN based on whether words like "auth", "connect", "socket" appear near the flagged line. For all TB-* rules, the assignment is unconditionally SAFETY regardless of context. For RC-003/RC-004, the rule's own subject words ("socket", "connect") drive the classification. 91% of SAFETY labels (186/205) are fixed by rule identity, not by instance-level judgment.

Rater. The rater is a ~60-line deterministic Python function (`classify_finding` in `precision_study_v2.py`), not a language model. The function is published alongside the paper. It does not constitute blind review and should not be cited as a precision or relevance measurement.

2.5 Threats to validity

1. **Co-location is not equivalence.** Some co-located findings detect unrelated patterns; some semantic overlaps occur on different lines.
2. **Ruff rule set understates overlap.** Only 30 of ruff's 800+ rules were tested. Known omissions include SIM115, B006, PGH004, S112, S502-504. Full ruff coverage would increase overlap.
3. **Suppression inflation.** Upstream `# noqa` and `# nosec` directives suppress ruff/bandit but not furqan-lint, inflating the not-co-located count. The size of this effect is not measured.
4. **Keyword classifier is not precision.** Q1 cannot detect false positives for 30 of 33 rules. Q2 is rule-category assignment for 91% of SAFETY labels. Neither constitutes a measurement.
5. **Test files inflate safety counts.** Installed packages include test directories. Test code contributes substantially to TB-026 (all 36 unique in RNG test suites), TB-023/024 (test trees), RC-003 (test utilities), and TB-032/033 (test data). The paper does not stratify by test/non-test.
6. **Non-Python findings.** 61 of 11,338 not-co-located findings come from C/C++ adapter rules (TB-029, TB-030, TB-032, TB-033) firing on numpy's bundled .c/.h files. These are unique by construction -- bandit and ruff cannot scan them.
7. **Convenience sample** of packages. Three packages (sqlalchemy, django, pandas) hold 54.7% of not-co-located findings.
8. **Single tool versions.**
9. **Conflict of interest.** The author develops furqan-lint. The analysis scripts were written in a Claude (Anthropic) sandbox. AI involvement is limited to script authorship; classification is deterministic.

---

3. Results

3.1 Three-way overlap

Category	Count	% of furqan	Description
UNIQUE	11,338	45.5%	No bandit or ruff finding within 2 lines
RUFF_ONLY	9,465	38.0%	ruff fires at same location, bandit does not
BANDIT_ONLY	264	1.1%	bandit fires at same location, ruff does not

Category	Count	% of furqan	Description
BOTH	3,866	15.5%	Both bandit and ruff fire at same location

45.5% of furqan-lint's findings are not co-located with either bandit or the 30-rule ruff subset. The rate is stable across line windows: 46.8% at window=0, 45.5% at 2, 43.7% at 5.

Per-package variation is wide: 18.0% (jinja2) to 92.1% (boto3), median 40.6%. Three packages -- sqlalchemy (3,857), django (1,355), and pandas (988) -- hold 54.7% of the not-co-located findings.

3.2 Per-rule composition

Full table of all 33 rules with at least one not-co-located finding, sorted by count:

Rule	Total	Not co-located	%	Detection target
SI-001	11,742	5,483	46.7%	Any type annotation (cf. ruff ANN401)
SI-005	2,794	2,252	80.6%	**kwargs passthrough
SI-007	2,017	1,791	88.8%	hasattr() swallows AttributeError
MR-005	964	829	86.0%	Empty except block
SI-003	269	262	97.4%	Blanket # noqa
MR-007	100	97	97.0%	try-except-continue in loop
RC-001	85	78	91.8%	open() without close
RC-004	73	69	94.5%	DB cursor without context manager
TB-018	128	58	45.3%	SQL injection via f-string
RC-003	53	53	100%	Socket/connection not closed
MR-002	455	51	11.2%	Broad except without re-raise
TB-004	52	51	98.1%	Mutable default argument
SI-004	1,894	40	2.1%	TODO/FIXME comment
RC-005	40	39	97.5%	Temp file without cleanup
TB-026	125	36	28.8%	Insecure random
TB-029	34	34	100%	C: sprintf buffer overflow
RC-002	20	19	95.0%	Lock acquire without release
TB-030	19	19	100%	C: non-literal format string
SI-002	666	18	2.7%	Blanket type: ignore
TB-023	14	14	100%	Insecure hash (MD5/SHA1)
TB-024	9	9	100%	Hardcoded secret
MR-003	10	8	80.0%	Nested try without re-raise
TB-033	4	4	100%	C++: C-style cast
TB-032	4	4	100%	C++: reinterpret_cast
MR-001	29	3	10.3%	Bare except without re-raise
TB-006	153	3	2.0%	Pickle/yaml deserialization
TB-025	4	3	75.0%	Insecure SSL context
TB-027	9	3	33.3%	Hardcoded password

Rule	Total	Not co-located	%	Detection target
TB-011	8	2	25.0%	Mutable default (dict)
TB-014	2	2	100%	Config from environment
TB-010	22	2	9.1%	Mutable default (list)
TB-005	115	1	0.9%	eval/exec
TB-017	38	1	2.6%	Hardcoded IP

Rules with 0% not-co-located: SI-006 (2,872 -- all co-located with B101), TB-020 (71 -- all bandit), TB-013 (6), TB-012 (5), TB-015 (1), TB-016 (1).

3.3 Exploratory classification (not a precision measurement)

The keyword classifier assigned categories to 585 sampled findings. These are the raw sample proportions, **not** population estimates (the sampling is disproportionate):

Category	Sample count	% of 581 Q1-positive	Stratum-weighted estimate
SAFETY	205	35.3%	~5.3% (~600 of 11,338)
HEALTH	333	57.3%	~77.6% (~8,800 of 11,338)
DESIGN	43	7.4%	~17.1% (~1,940 of 11,338)
Q1-negative	4	(excluded)	--

The sample overrepresents small TB/RC strata (sampled at 35-100%) relative to large SI strata (sampled at <1%). The stratum-weighted estimate applies each rule's sample rate to its population count. SAFETY is driven almost entirely by rule category (TB-* = SAFETY unconditionally), not instance-level context analysis.

Q1 reported 581/585 as "pattern present," but this is uninformative: the classifier can only flip 3 of 33 rules to false positive. Precision remains unmeasured.

---

4. Discussion

4.1 Composition of not-co-located findings

The 11,338 not-co-located findings break into four groups by rule family:

SI-* (scan-incomplete): 9,846 findings (86.8%). Dominated by SI-001 (5,483 -- Any annotations, with a direct ruff counterpart in ANN401; the 46.7% not-co-located portion reflects a scope difference: ANN401 flags Any on function signatures, while SI-001 flags Any anywhere including generics like `dict[str, Any]`). SI-005 (2,252 -- kwargs passthrough) and SI-007 (1,791 -- hasattr masking) have no known ruff or bandit counterpart. SI-003 (262 -- blanket noqa) partially overlaps PGH004, which was not included in the ruff selection.

MR-* (error handling): 988 findings (8.7%). MR-005 (829 -- empty except) partially overlaps ruff BLE001/E722 and bandit B110; the 86% not-co-located rate for MR-005 reflects the remaining findings where bandit/ruff's narrower rules do not fire. MR-002 (51 -- broad except, 11.2% not-co-located) is substantially covered by ruff BLE001.

RC-* (resource lifecycle): 258 findings (2.3%). RC-001 (open without close), RC-003 (connection not closed), RC-004 (cursor without context manager), RC-005 (temp file without cleanup). ruff's SIM115 was not included in the selection and would increase overlap for RC-001. No bandit counterpart exists for any RC rule.

TB-* (trust boundary): 246 findings (2.2%). Mixed: 61 are non-Python (C adapter rules unique by construction), the remainder are small counts across 16 rules, most substantially covered by bandit (TB-005, TB-006, TB-017, TB-020, TB-021).

4.2 What the data shows and does not show

Shows: furqan-lint and the 30-rule ruff subset produce findings at different locations 45.5% of the time. The not-co-located findings concentrate in SI-005 (kwargs), SI-007 (hasattr), MR-005 (empty except), and RC-* (resource lifecycle) -- rule families without direct bandit or ruff counterparts in the selection.

Does not show: Whether those findings are true positives (precision is unmeasured), whether they matter for safety (the keyword classifier is rule-category assignment, not instance-level judgment), or whether they are unique against ruff's full rule set (only 30 of 800+ rules tested).

4.3 SI-001 dominance

SI-001 (Any annotation) is 48.4% of all not-co-located findings. It has a direct counterpart in ruff ANN401. The 46.7% not-co-located rate is explained by a scope difference (SI-001 flags Any in all contexts; ANN401 flags it only on function signatures), not by genuinely different detection. A full accounting of the tool's novel contribution should exclude SI-001 or at minimum report it separately.

Excluding SI-001, the not-co-located pool is 5,855 findings (23.5% of total), dominated by SI-005 (2,252), SI-007 (1,791), and MR-005 (829).

4.4 Limitations

1. **Precision unmeasured.** The keyword classifier cannot detect false positives for 30 of 33 rules. No blind human review was performed.
2. **Ruff subset understates overlap.** 30 of 800+ rules tested. Known missing mappings (SIM115, B006, PGH004, S112, S502-504) would increase overlap for RC-001, TB-004, SI-003, MR-007, TB-025, TB-006.
3. **Suppression inflation.** `# noqa` and `# nosec` suppress ruff/bandit but not furqan-lint. Not measured.
4. **Keyword classifier is rule-category assignment.** 91% of SAFETY labels are fixed by rule prefix (TB = SAFETY), not by code context. The stratum-weighted ~5.3% safety rate is itself unreliable.
5. **Test code not excluded.** TB-026 (all 36 in test RNG suites), RC-003 (14/20 sampled in test utilities), TB-023/024 (test trees) inflate counts.
6. **Non-Python findings.** 61 C/C++ adapter findings are unique by construction.
7. **Package concentration.** sqlalchemy + django + pandas = 54.7% of not-co-located findings.
8. **Single versions, convenience sample, one-directional decomposition.**
9. **COI.** Author develops furqan-lint. Scripts written in Claude (Anthropic) sandbox.

---

5. Conclusion

45.5% of furqan-lint's findings are not co-located with bandit or a 30-rule ruff subset (within 2 lines in the same file). The largest component is SI-001 (Any annotation, 48.4% of not-co-located), which has a direct ruff counterpart and is not-co-located due to a scope difference. Excluding SI-001, 5,855 findings (23.5% of total) are not co-located, primarily from rules without known counterparts in the 30-rule selection: SI-005 (kwargs passthrough), SI-007 (hasattr masking), MR-005 (empty except), and RC-* (resource lifecycle).

Precision is not measured: the exploratory classifier cannot detect false positives for most rules. The stratum-weighted relevance estimate (~5.3% safety, ~77.6% health, ~17.1% design) is driven by rule-category assignment, not instance-level judgment, and should not be cited as a safety measurement.

Whether the not-co-located findings from rules without known counterparts represent a distinct and useful analysis class requires a precision study with blind human raters and a comparison against ruff's full rule set -- neither of which is performed here. The raw data to answer both questions is shipped.

Open questions for future work:

- Blind human precision study (2+ raters, inter-rater kappa)
- Full ruff rule set comparison (800+ rules)
- Test-path stratification
- semgrep inclusion as a fourth tool
- Reverse decomposition: what do bandit and ruff find that furqan-lint misses?

Data availability

All raw data is in the repository under `furqan-lint/paper/`: raw_json/ (furqan-lint, 24,933 findings), bandit_json/ (bandit, 29,959), ruff_json/ (ruff, 44,855), overlap/ (two-way and three-way decomposition CSVs and summaries), precision_v2/ (585 classified findings with source context and classifier source code), benchmark_v2.1.0.csv, versions.json, rule_manifest.json. Scripts: benchmark.py, three_way_decomposition.py, precision_study_v2.py, run_ruff.py. Tools: furqan-lint v2.1.0, bandit 1.9.4, ruff 0.16.9, Python 3.13.18. Note: the scripts reference absolute paths from the original execution environment; adjust BASE and SITE_PACKAGES paths for local reproduction.

The tool is open-source under Apache-2.0: https://github.com/BayyinahEnterprise/Kun-Fayakun

---

References

[1] B. S. Arfeen, "furqan-lint vs. bandit: A Finding-Count Comparison Across 25 Python Codebases," BayyinahEnterprise, 2026. https://github.com/BayyinahEnterprise/Kun-Fayakun

[2] PyCQA, "Bandit: Security linter for Python," v1.9.4, https://bandit.readthedocs.io/

[3] Astral Software, "Ruff: An extremely fast Python linter," v0.16.9, https://docs.astral.sh/ruff/
